Lab4 : Asymmetric Ciphers

Exercise 1 : OpenSSL Asymmetric encryption with RSA

1. Generate a key pair (public, private) for Alice and save it in a file named `key.pem`. What is the encoding used in this file ?
2. How can you extract the public key saved in `key.pem` and store it in a file named `pub.pem`?
3. Show and explain the content of `pub.pem`.
4. Bob, who has access to the file `pub.pem`, wants to encrypt a secret message and send it to Alice using Alice's public key. He sends the encrypted message to Alice. What command should he use?
5. What command allows Alice to decrypt the secret using her private key ?

Exercice 2 : Electronic certificates

1. What is an electronic certificate and what is its purpose ?
2. Retrieving, viewing, and transcoding certificates:
    a. Enter the following script **`get-cert.sh`**:

```
#!/bin/sh
# usage: get-cert.sh remote.host.name [port]
REMHOST=$1
REMPORT=${2:-443}
echo |\
openssl s_client -connect ${REMHOST}:${REMPORT} 2>&1 |\
sed -ne '/-BEGIN CERTIFICATE-/,/-END CERTIFICATE-/p'
```

    b. Explain the role of the `openssl s_client` command and the `sed` command.
    c. Execute this script to retrieve the certificate of an Internet website of your choice, then display this certificate (which is in PEM format).
    d. Convert this certificate from PEM format to DER format.
    e. What is the difference between these two formats? What other formats are possible?
3. Creation of a self-signed X.509 certificate: Generate a public/private key pair, then create a certificate signing request (CSR). Next, use this information to generate a self-signed certificate.
4. Install an Apache https server, then use a script of your choice to test its security and obtain a score. Try to increase this score.

Exercise 3 : DH in Openssl

1. Generate DH Parameters

```
openssl dhparam -out dhparams.pem 2048
```

    a. What is the purpose of the file `dhparams.pem`?
    b. What kind of data does it contain?
2. Generate Alice's and Bob's DH key pairs. What do the `-paramfile` and `-genpkey` options do?

```
Alice # openssl genpkey -paramfile dhparams.pem -out alice_priv.pem
Bob # openssl genpkey -paramfile dhparams.pem -out bob_priv.pem
```

3. Extract their public keys :

```
openssl pkey -in alice_priv.pem -pubout -out alice_pub.pem
openssl pkey -in bob_priv.pem -pubout -out bob_pub.pem
```

What is the difference between alice_priv.pem and alice_pub.pem?

4. Derive the shared secret

```
# Alice computes the shared secret
openssl pkeyutl -derive -inkey alice_priv.pem -peerkey bob_pub.pem -out alice_secret.bin
# Bob computes the shared secret
openssl pkeyutl -derive -inkey bob_priv.pem -peerkey alice_pub.pem -out bob_secret.bin
```

What is the purpose of the -derive command?

5. Compare the shared secrets

```
cmp alice_secret.bin bob_secret.bin && echo "Shared secret matches!" || echo "Mismatch!"
```

6. Convert the binary shared secret to a hexadecimal or base64-encoded string.

Challenge 1: Factorization

1. Factor the integer :
   $n = 3148240326296492491829836036538028522262397298543021512290073$
   using the method of **successive divisions** (trial division).
2. Same question for :
   $n = 4433634977317959977189716351978918572296527677331175210881861$
3. **Fermat method :** is an integer factorization method that attempts to factor an odd integer n by expressing it as the difference of two squares: $n = a^2 - b^2$. Letting $n = \alpha\beta$, where α is the largest divisor of n less than $\sqrt{n}$, the number of arithmetic operations is on the order of: $\left(\sqrt{n} - \alpha\right)^2 / 2\alpha$. In particular, if n has a divisor close to $\sqrt{n}$, Fermat's method is faster than trial division. However, in the worst case, it may require up to n arithmetic operations.
   Using this method, factorize the number :
   $n = 4433634977317959977189716351978918572296527677331175210881861$

Challenge 2 : Root-me : Decrypt the password

The validation password was encrypted using public key `pubkey.pem` :


```
e8oQDihsmkvjT3sZe+EE8lwNvBEsFegYF6+OOFOiR6gMtMZxxba/bIgLUD8pV3yEf0gOOfHuB5bC3vQmo7bE4PcIKfpFGZBA
```